

FACULDADE DE TECNOLOGIA SENAI FÉLIX GUIARD

CURSO SUPERIOR DE TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

RELATÓRIO TÉCNICO: PROJETO CONEXÕES SEGURAS (PARTE 1/3)

Equipe de Desenvolvimento: Davi Marcondes Paes de Souza, Renan Ramos Capeleti, Kauan Alejandro da Rosa, João Gabriel Floriano Olimpio

Disciplina: Tecnologia da Informação e Conectividade

Instituição: SENAI Félix Guisard

INTRODUÇÃO

Este relatório técnico descreve de forma estruturada e aprofundada o desenvolvimento da primeira parte (1/3) do projeto semestral interdisciplinar de Conexões Seguras. O objetivo central deste projeto reside na compreensão prática e fundamentação teórica de três módulos essenciais que governam a segurança das comunicações digitais em redes corporativas modernas: a interceptação de tráfego local (sniffing), a infraestrutura de chaves de transporte (handshake TLS) e a gestão segura de identidade de sessão (cookies).

Para cada um dos módulos abordados, este relatório detalha a base conceitual de infraestrutura afetada e apresenta uma Prova de Conceito (PoC) prática executada de forma realista em ambiente de nuvem (Ubuntu Server em instância AWS EC2), mapeando de forma rigorosa as ferramentas utilizadas, o passo a passo de execução e a análise explicativa dos resultados para validação acadêmica.

MÓDULO 1: SNIFFING E INTERCEPTAÇÃO (CAMADAS 2 E 3)

Ferramenta Utilizada: Utilitário Unix nativo de rede tcpdump para escuta e filtragem de pacotes brutos do barramento, e o utilitário netcat (nc) como ouvinte ativo na interface física.

Passo a Passo de Execução:

1. No console do Ubuntu AWS, identificar o nome da placa de rede ativa (ex: ens5) por meio do comando `ip link show`.
2. Alterar o driver da NIC para o modo promíscuo executando o comando administrativo:
`sudo ip link set dev ens5 promisc on`
3. Confirmar a mudança de estado verificando a presença da flag PROMISC na saída de `ip link show ens5`.
4. Para evitar conexões recusadas no cliente, abrir uma escuta TCP ativa no Ubuntu em background na porta 8080:
`sudo nc -lk 8080`
5. Em outra janela do terminal do Ubuntu, iniciar a captura e formatação legível (ASCII) dos pacotes que passarem pela porta 8080:
`sudo tcpdump -i ens5 -A -s 0 'tcp port 8080'`
6. No terminal do computador local (cliente), simular o envio de um formulário de login em texto claro enviando

uma requisição HTTP POST para a AWS:

```
curl -X POST -d "user=admin&pass=senai123" http://IP_PUBLICO_DA_AWS:8080/login
```

Resultado Obtido:

O resultado obtido consistiu na interceptação e captura bem-sucedida do tráfego de rede direcionado à porta 8080 da instância AWS por meio do utilitário tcpdump. Durante a análise do fluxo capturado, foi possível isolar a requisição HTTP POST enviada pelo cliente externo, contendo todos os cabeçalhos da camada de aplicação e o corpo da mensagem perfeitamente visível. As imagens e capturas de tela contendo os logs detalhados do terminal e as linhas de código capturadas foram devidamente arquivadas e podem ser consultadas na pasta de evidências do projeto.

Explicação Técnica do Resultado: Uma vez que a interface de rede ens5 foi colocada em modo promíscuo, a filtragem de frames de hardware realizada pelo chip físico da controladora de rede (NIC) foi desabilitada. Sob condições operacionais normais, a placa de rede descartaria qualquer frame cujo endereço MAC de destino não correspondesse ao seu próprio endereço físico gravado em hardware, a menos que fosse tráfego de broadcast ou multicast. Ao forçar a flag PROMISC no kernel Unix, a filtragem da Camada 2 é ignorada, empurrando todos os frames brutos para o buffer do sistema operacional. Como o protocolo HTTP trafega na camada de aplicação sem envelopamento criptográfico de transporte, os bytes contidos no segmento TCP foram remontados e convertidos diretamente em caracteres ASCII legíveis, expondo as credenciais sensíveis (user=admin&pass=senai123) do formulário em texto claro.

MÓDULO 2: CRIPTOGRAFIA E HANDSHAKE (A BASE DO TLS)

Ferramenta Utilizada: Utilitário criptográfico de linha de comando openssl nativo de sistemas Unix para geração de entropia, chaves assimétricas RSA e implementação de cifra simétrica AES-256-CBC.

Passo a Passo de Execução:

1. No terminal do Ubuntu, simular as chaves do servidor gerando uma chave privada RSA de 2048 bits:
`openssl genrsa -out server_private.pem 2048`
2. Extrair a chave pública correspondente (que seria distribuída via certificado digital):
`openssl rsa -in server_private.pem -pubout -out server_public.pem`
3. Do lado do cliente, simular a criação de uma chave simétrica de sessão de 256 bits gerada aleatoriamente:
`openssl rand -hex 32 > session_key.txt`
4. Cifrar a chave de sessão usando a chave pública RSA do servidor (simulando a etapa segura do handshake do navegador):
`openssl pkeyutl -encrypt -pubin -inkey server_public.pem -in session_key.txt -out encrypted_key.bin`
5. Visualizar o envelope criptográfico gerado utilizando o leitor de binários:
`xxd encrypted_key.bin`
6. Decifrar a chave de sessão no servidor com sua respectiva chave privada RSA:
`openssl pkeyutl -decrypt -inkey server_private.pem -in encrypted_key.bin -out decrypted_key.txt`
7. Com a chave de sessão simétrica estabelecida em ambas as pontas, cifrar o arquivo confidencial de payload utilizando criptografia AES-256-CBC de alta velocidade:
`echo "DADOS_CONFIDENCIAIS_DO_USUARIO_SENAI_FELIX_GUISARD" > dados.txt`
`openssl enc -aes-256-cbc -salt -in dados.txt -out seguro.enc -pass file:decrypted_key.txt -pbkdf2`
8. Exibir o tráfego ilegível que passará pelo barramento:
`cat seguro.enc`

Resultado Obtido:

O resultado obtido demonstrou o funcionamento completo do ciclo criptográfico híbrido. A chave simétrica temporária gerada pelo cliente foi envelopada de forma assimétrica com a chave pública do servidor e, após o envio, foi decifrada com sucesso utilizando a chave privada correspondente, gerando dois valores idênticos em ambos os lados e confirmando o estabelecimento do segredo compartilhado. Na sequência, o payload confidencial de dados foi cifrado utilizando o algoritmo AES-256-CBC, gerando um arquivo de saída completamente ilegível e protegido. Todas as saídas textuais detalhadas, chaves geradas e criptogramas em formato hexadecimal foram registrados e estão armazenados na pasta de evidências..

Explicação Técnica do Resultado: O teste prático simula a arquitetura de criptografia híbrida empregada pelo TLS/HTTPS para mitigar restrições severas de latência de rede e processamento de CPU. Algoritmos assimétricos (como RSA) envolvem exponenciação modular de inteiros massivos baseados na dificuldade matemática de fatoração de produtos de números primos extensos. Esse nível de complexidade matemática exige alta capacidade da CPU, tornando inviável cifrar fluxos contínuos de megabytes na internet. A solução híbrida realiza o handshake TLS utilizando a cifra assimétrica exclusivamente para estabelecer e sincronizar com total confidencialidade uma chave simétrica efêmera (AES-256) criada pelo cliente. Uma vez que o segredo simétrico é compartilhado entre cliente e servidor, a transmissão de arquivos pesados passa a ser processada pelo algoritmo AES. Isso garante alta performance computacional e proteção integral, tirando proveito das instruções de aceleração por hardware nativas (AES-NI) contidas nas CPUs modernas.

MÓDULO 3: SEGURANÇA NA CAMADA DE APLICAÇÃO (DESENVOLVIMENTO)

Ferramenta Utilizada: Ambiente de runtime Node.js com o framework web corporativo Express para subida de uma API de login real na porta 3000, e o utilitário local curl para análise dos cabeçalhos HTTP de resposta.

Passo a Passo de Execução:

1. No Ubuntu, estruturar e salvar o arquivo `app_sessao.js` contendo as rotas de login inseguro (sem restrição de cookies) e seguro (com diretivas de hardening).
2. Inicializar a aplicação Node.js:

```
npm init -y && npm install express  
node app_sessao.js
```
3. No computador local (cliente), simular a requisição de autenticação para a rota vulnerável e examinar a resposta com os cabeçalhos ativos:

```
curl -i http://IP_PUBLICO_DA_AWS:3000/login-inseguro.
```
4. Repetir o teste disparando a chamada para a rota segura para analisar as tags adicionais:

```
curl -i http://IP_PUBLICO_DA_AWS:3000/login-seguro.
```

Resultado Obtido:

O resultado obtido evidenciou o contraste comportamental entre uma aplicação vulnerável e uma aplicação protegida por meio de cabeçalhos de resposta HTTP. Na rota de testes insegura, o servidor Express retornou o cookie de sessão de forma totalmente exposta, desprovido de qualquer atributo restritivo. Por outro lado, ao efetuar a chamada para a rota de hardening, o servidor aplicou com sucesso os parâmetros protetivos, anexando explicitamente as diretivas de controle de segurança de sessão. A exibição visual completa das estruturas dos cabeçalhos de resposta HTTP recebidos pelo cliente curl foi documentada e arquivada na pasta de evidências

Explicação Técnica do Resultado: Devido à natureza inerentemente apátrida (stateless) do protocolo HTTP, o

controle de estado e a manutenção de sessões autenticadas dependem do tráfego de chaves de identificação geradas pelo servidor (Cookie SessionID). A negligência na configuração destas chaves expõe o ecossistema a ataques críticos de Session Hijacking (Sequestro de Sessão) via Cross-Site Scripting (XSS). Sem defesas, um script malicioso injetado no navegador executa a chamada `document.cookie`, coletando o token e enviando-o para servidores de terceiros. A inclusão do atributo `HttpOnly` altera esse comportamento, ordenando que o engine de renderização do navegador remova e blinde o cookie de qualquer acesso do DOM via scripts client-side. Adicionalmente, a diretiva `Secure` força o navegador a transmitir o cookie exclusivamente sob requisições criptografadas com TLS (HTTPS). Sob tráfego HTTP simples, como demonstrado no Módulo 1, os cabeçalhos trafegam limpos no barramento, permitindo que sniffers capturem e clonem as contas locais instantaneamente. O HTTPS atua envelopando toda a mensagem HTTP antes da injeção física na rede, mantendo os identificadores confidenciais.

CONCLUSÃO GERAL

A realização das atividades práticas e simulações do projeto de Conexões Seguras (Parte 1/3) permitiu à equipe de estudantes de Análise e Desenvolvimento de Sistemas compreender o encadeamento e o papel de diferentes mecanismos e diretivas de defesa ao longo do Modelo OSI. Os testes reais realizados nos terminais do Ubuntu Server instanciado na AWS comprovaram o perigo inerente do tráfego sem criptografia de transporte, ao mesmo tempo em que validaram os processos práticos de compartilhamento de segredos simétricos (TLS) e o uso obrigatório das diretivas de cookies (`HttpOnly` e `Secure`) para a blindagem de credenciais administrativas contra vazamentos e interceptações locais.